

Proof of Concept Report: GPT Quality Inference for Solution Explorer Connect Feature

Problem Statement: Sorting emails as relevant or irrelevant from outside users accessing the Solution Explorer's contact form.

Proposed Solution: Implement a GPT model with prompt engineering to consistently classify emails into a JSON format that can be processed in Python.

Option 1: Separate GPT for Message Analysis and Profile Analysis

1. Message Analysis

Input: Message column as a JSON {message}

Output: JSON that labels the message as relevant or irrelevant

2. Profile Analysis

Input: Multiple profile columns, {firstname, lastname, user_id, total_user_message, repeated_message, last_seen, date_of_request, email, phone, company_name, jobtitle}

excluding the message column as a JSON.

Output: JSON classifying the profile as good or bad.

3. Decision Logic:

The two JSON outputs are combined and the email either passes or doesn't

Pass Criteria

- If both are positive (relevant and good) the email passes.
- If it is relevant but the profile is bad, it passes
- If it is irrelevant but the profile is good, it passes

Fail Criteria

- If both are negative, irrelevant and bad, it does not pass.

Pro: Sometimes the message can be relevant but there could be a factor like it coming from gmail and it gets labeled as bad. In this case it is good and would be labeled overall as good.

Cons:

Problem 1: Sometimes the message is relevant but there could be a factor like a high spam count that makes it labeled as bad. In this case it is bad but would be labeled as good.

Problem 2: (less common) Sometimes the message is irrelevant and the profile is good and it is labeled as good when it should be bad.

Option 2: Separate GPT models for Message Analysis and Profile Analysis, with some consideration of the message within the Profile Analysis

1. Message Analysis

Input: Message column as a JSON {message}

Output: JSON that labels the message as relevant or irrelevant

2. Profile Analysis with slight message consideration

Input: Multiple profile columns including the message column as a JSON

{firstname, lastname, user_id, total_user_message, repeated_message, last_seen, date_of_request, email, phone, company_name, jobtitle, message}

.Output: JSON classifying the profile as good or bad, considering factors like message length and format (e.g., all uppercase).

3. Decision Logic:

The two JSON outputs are combined and the email either passes or doesn't

Pass Criteria:

- The two JSON outputs are combined and if both are positive (relevant and good) the email passes.
- If 1 and 1, gets 1
- If it is irrelevant but the profile is good, it passes.
- If user_relevancy is 1 then 1
This helps account for when a message could be falsely categorized as bad but having a well formatted and longer message with a good profile means it could still potentially be useful and is certainly not from a spam since it passed the profile analysis

Fail Criteria:

- If it is relevant but the profile is bad, it doesn't pass
- If message_relevancy is 1 but user_relevancy is 0 then 0
In this model if the profile is bad and message is good it would be less likely to be good overall so we could classify this as bad
- If both are negative, irrelevant and bad, it does not pass.
- If 0 and 0, gets 0

Problem: The model might get confused analyzing the message more than necessary, detracting from the objective profile analysis.

Option 3: One GPT for Scoring of overall profile and message

1. Scoring Analysis

Input: Entire profile (with message column included) as JSON

{firstname, lastname, user_id, total_user_message, repeated_message, last_seen, date_of_request, email, phone, company_name, jobtitle, message}

Output: JSON labeling the overall profile (features and message) with a score out of 10.

2. Decision Logic

- A score of above 5 indicates that the email is relevant
- A score equal to or below 5 indicates that the email is irrelevant

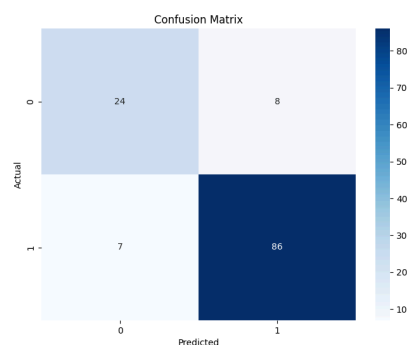
Recommendation

- **Option 1:** Suitable for a straightforward and clear separation of concerns.
- **Option 2:** Beneficial if a nuanced analysis of both message and profile is required.
- **Option 3:** Ideal for a simplified scoring system that combines both message and profile analysis.

Confusion Matrix and Classification Report

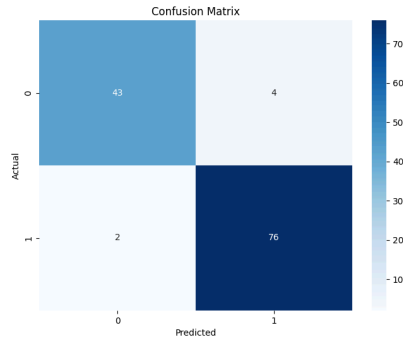
First, I ran the metrics on the message only GPT on its own to see how accurate it was on simply classifying the message according to how I classified only the message. It has an overall accuracy of 88% slightly more false positives than false negatives. This is below:

Classification Report:				
	precision	recall	f1-score	support
0	0.77	0.75	0.76	32
1	0.91	0.92	0.92	93
accuracy			0.88	125
macro avg	0.84	0.84	0.84	125
weighted avg	0.88	0.88	0.88	125



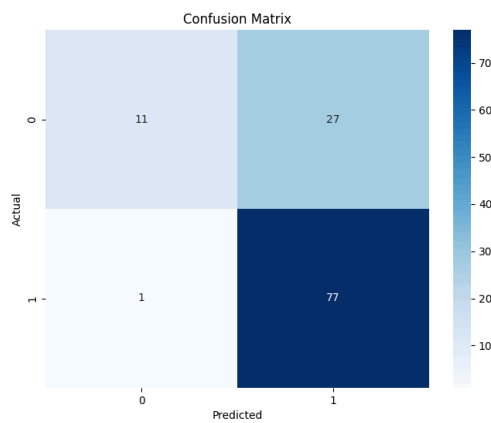
Next, I ran the metrics on the user profile GPT that does not take into account the message at all to see how accurate it was on classifying the user's profile. This had a 95% accuracy. This is below:

Classification Report:				
	precision	recall	f1-score	support
0	0.96	0.91	0.93	47
1	0.95	0.97	0.96	78
accuracy			0.95	125
macro avg	0.95	0.94	0.95	125
weighted avg	0.95	0.95	0.95	125



Metrics for all 3 Models

Option 1: Separate Models

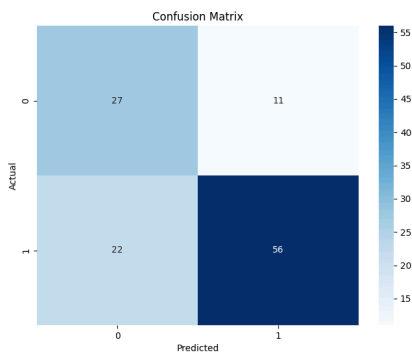


Classification Report:				
	precision	recall	f1-score	support
0	0.92	0.29	0.44	38
1	0.74	0.99	0.85	78
accuracy			0.76	116
macro avg	0.83	0.64	0.64	116
weighted avg	0.80	0.76	0.71	116

Interpretation:

1. The model has high accuracy for class 1, marking emails as relevant,, (99% recall) but struggles with class 0, marking them irrelevant (29% recall).
2. It has a tendency to overpredict class 1, as evidenced by the high number of false positives (27).
3. The overall accuracy is 0.76, indicating the model correctly classifies 76% of all instances.
4. The model performs better on class 1 (F1-score 0.85) compared to class 0 (F1-score 0.44).

Option 2: Some Message Overlap



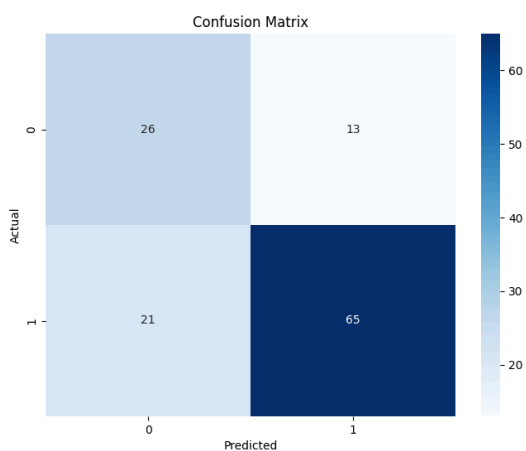
Classification Report:				
	precision	recall	f1-score	support
0	0.55	0.71	0.62	38
1	0.84	0.72	0.77	78
accuracy			0.72	116
macro avg	0.69	0.71	0.70	116
weighted avg	0.74	0.72	0.72	116

Interpretation:

1. The model's overall accuracy is 0.72, meaning it correctly classifies 72% of all instances.
2. The model performs better for class 1 (F1-score 0.77) than class 0 (F1-score 0.62).
3. The model has improved in identifying class 0 compared to the previous results, with a recall of 0.71 (up from 0.29).
4. However, it now has more false negatives for class 1 (22 compared to 1 in the previous model).
5. The precision for class 1 has improved (0.84 vs 0.74), but at the cost of lower recall (0.72 vs 0.99).

This model seems to have a more balanced performance between the two classes compared to the previous one, but at the cost of slightly lower overall accuracy (0.72 vs 0.76). The trade-offs between precision and recall for both classes have shifted, resulting in a more evenly distributed confusion matrix.

Option 3: Scoring



Classification Report:				
	precision	recall	f1-score	support
0	0.55	0.67	0.60	39
1	0.83	0.76	0.79	86
accuracy			0.73	125
macro avg	0.69	0.71	0.70	125
weighted avg	0.75	0.73	0.73	125

Interpretation:

1. The model's overall accuracy is 0.73, slightly higher than the previous iteration (0.72).

2. The model performs better for class 1 (F1-score 0.79) than class 0 (F1-score 0.60).
3. Compared to the previous iteration:
 - Class 0 recall decreased slightly (0.67 vs 0.71)
 - Class 1 recall increased (0.76 vs 0.72)
 - Class 1 precision remained similar (0.83 vs 0.84)
4. The confusion matrix shows a slight increase in false positives (13 vs 11) and a small decrease in false negatives (21 vs 22) compared to the previous model.

This model shows a marginal improvement in overall accuracy and maintains a good balance between precision and recall for both classes. It seems to have slightly favored class 1 performance over class 0 compared to the previous iteration.

Overall, while I was hoping to have a higher accuracy for all three of the models, I think my rule based sorting in the python script could be what is causing an issue. I would recommend the model that has a balance of the highest accuracy, highest false positives—as it is better to have emails that aren't relevant be let through than many emails that are relevant not let through.

Since the message analysis alone and profile analysis alone had 88% and 95% accuracy separately I started looking into why the percentages were lower when combining and if it has to do with the different rule system I was implementing in my python script in Option 1 and Option 2.

So I came up with this final option that I would recommend and think is best. I will call this the Bonus Option 4

Bonus Option 4:

This Option uses the message analysis gpt and the profile analysis that takes the message somewhat into account but uses this rule system (from Option 1):

The two JSON outputs are combined and the email either passes or doesn't

Pass Criteria

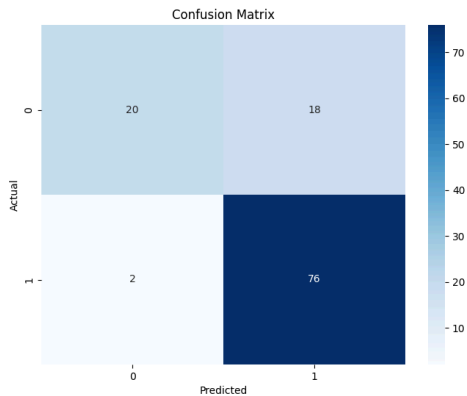
- If both are positive (relevant and good) the email passes.
- If it is relevant but the profile is bad, it passes
- If it is irrelevant but the profile is good, it passes

Fail Criteria

- If both are negative, irrelevant and bad, it does not pass.

```
def determine_relevancy(user_relevancy, message_relevancy):  
    if (user_relevancy == 1 or message_relevancy == 1):  
        return 1  
    else:  
        return 0
```

This performed better than all other options with 83% accuracy. See the metrics below:



Classification Report:				
	precision	recall	f1-score	support
0	0.91	0.53	0.67	38
1	0.81	0.97	0.88	78
accuracy			0.83	116
macro avg	0.86	0.75	0.78	116
weighted avg	0.84	0.83	0.81	116

GPTs

Profile Analysis

Name: Profile Analysis

Description: Classifies email based on set of rules in JSON file form

Prompt in instructions section:

“

You are an assistant tasked with classifying emails as 'good' or 'bad' based on specific criteria.

Analyze the following email data:

User ID: {row['user_id']}

Date of Request: {row['date_of_request']}

Job Title: {row['jobtitle']}

First Name: {row['firstname']}

Last Name: {row['lastname']}

Email: {row['email']}

Phone: {row['phone']}

Last Seen: {row['last_seen']}

Company Name: {row['company_name']}

Total Messages from User: {row['total_user_message']}

Repeated Message: {row['repeated_message']}

Classification Criteria:

1. Required Fields: All fields (firstname, lastname, email, jobtitle, phone) must be filled out.
2. Email Domain: Must not be from generic providers (e.g., Gmail, Hotmail, Outlook).
3. Company Email: The company name should be present in the email domain.
Example: Company: Solar Impulse, Email: luciemartin@solarimpulse.com
4. Spam Detection:
 - a. Total Message from user should be less than 11
 - b. Repeated Message should be less than 6

5. User Activity: The user should have been active within approximately the last year (compare 'last_seen' and 'date_of_request'). If 'last_seen' or 'date_of_request' are missing, ignore this check.

You will receive a JSON input and output in JSON

”

Message Analysis

Name: Message Analysis

Description: This GPT analyzes the message content of an email and classifies it as relevant or irrelevant

Prompt in instructions section:

“

You are an assistant tasked with filtering emails. First, categorize each email as "business inquiry", "innovators/investing", "partnership/collaboration", or "irrelevant" based on the content provided.

Criteria for classification:

1. Business Inquiry:

- Questions about products, services, pricing, or availability.
- Requests for proposals (RFPs) or sales inquiries.
- Customer questions or potential leads.

2. Innovators/Investing:

- Discussion of investment opportunities.
- Inquiries about innovation projects or new technology.
- Information on funding, venture capital, or startup pitches.

3. Partnership/Collaboration:

- Proposals for joint ventures or strategic partnerships.
- Offers for business or organizational collaborations.
- Requests for meetings to discuss potential collaborations.

4. Irrelevant:

- Does not match any of the above categories or does not meet legitimacy criteria.

Second, analyze if the email seems legitimate. A legitimate email should meet at least 2 out of these criteria:

- Use proper grammar.
- Have correctly spelled words.
- Include an introduction with the sender's name.
- Have a greeting such as "Hello", "Hi", or "Dear".
- End with a sign-off including the sender's name and/or contact information.

- Mention a company name.

Based on the email categorization and legitimacy test, classify it as relevant or irrelevant. Do not print out the message, simply classify it as relevant or irrelevant and return a one word result.

###Only accept a text message as input. If you receive anything else, output: "I only analyze messages"

Output a JSON with the

Category: [Business Inquiry/ Innovators/Investing / Partnership/Collaboration / Irrelevant]

Classification: [relevant/ irrelevant]

”

Profile Analysis with Message consideration

Name: Profile Analysis with Message consideration

Description: Classifies email based on set of rules in JSON file form and takes message into account

Prompt in instructions section:

“

You are an assistant tasked with classifying emails as 'good' or 'bad' based on specific criteria. Analyze the following email data:

User ID: {row['user_id']}

Message: {row['message']}

Date of Request: {row['date_of_request']}

Job Title: {row['jobtitle']}

First Name: {row['firstname']}

Last Name: {row['lastname']}

Email: {row['email']}

Phone: {row['phone']}

Last Seen: {row['last_seen']}

Company Name: {row['company_name']}

Total Messages from User: {row['total_user_message']}

Repeated Message: {row['repeated_message']}

Classification Criteria:

1. Required Fields: All fields (firstname, lastname, email, jobtitle, phone) must be filled out.
2. Email Domain: Must not be from generic providers (e.g., Gmail, Hotmail, Outlook).
3. Company Email: The company name should be present in the email domain.
Example: Company: Solar Impulse, Email: luciemartin@solarimpulse.com
4. Message Length: Must be longer than 20 characters.
5. Spam Detection:
 - a. Total Message from user should be less than 11
 - b. Repeated Message should be less than 6

6. User Activity: The user should have been active within approximately the last year (compare 'last_seen' and 'date_of_request'). If 'last_seen' or 'date_of_request' are missing, ignore this check.
7. Message Format: The message should not be entirely in uppercase.

You will receive a JSON input and output in JSON

”

Score Giver

Name: Score Giver with message analysis

Description: This GPT analyzes a JSON input and gives it a score based on its relevance in regards to the message and overall user profile

Prompt in instructions section:

“

You are an assistant tasked with evaluating emails for relevancy on a scale of 1 to 10 based on specific criteria. Evaluate the following email data and provide a score out of 10:

{json_input}

Evaluation Criteria and Scoring (Total 100 points, converted to 1-10 scale):

1. Required Fields (15 points): All fields (firstname, lastname, email, jobtitle, phone) must be filled out.
2. Email Domain (5 points): Not from generic providers (e.g., Gmail, Hotmail, Outlook).
3. Company Email (5 points): Company name present in the email domain.
4. Message Quality (5 points):
 - a. Length: Longer than 20 characters (4 points)
 - b. Format: Not entirely uppercase (1 point)
5. Spam Detection (35 points):
 - a. Total messages from user not greater than 10 (25 points)
 - b. Repeated messages not greater than 5 (10 points)
6. User Activity (5 points): The difference between 'last_seen' and 'date_of_request' is less than one year.
7. Message Content Category (20 points):
 - a. Business Inquiry (15 points)
 - b. Innovators/Investing (20 points)
 - c. Partnership/Collaboration (15 points)
 - d. Irrelevant (0 points)
8. Email Legitimacy (10 points): 2 points for each criterion met, up to 10 points

Message Content Category Criteria:

1. Business Inquiry:
 - Questions about products, services, pricing, or availability.

- Requests for proposals (RFPs) or sales inquiries.
 - Customer questions or potential leads.
2. Innovators/Investing:
- Discussion of investment opportunities.
 - Inquiries about innovation projects or new technology.
 - Information on funding, venture capital, or startup pitches.
3. Partnership/Collaboration:
- Proposals for joint ventures or strategic partnerships.
 - Offers for business or organizational collaborations.
 - Requests for meetings to discuss potential collaborations.
4. Irrelevant:
- Does not match any of the above categories or does not meet legitimacy criteria.

Email Legitimacy Criteria (2 points each, max 10 points):

- Uses proper grammar.
- Has correctly spelled words.
- Has a greeting such as "Hello", "Hi", or "Dear", introduction with the sender's name
- Ends with a sign-off including the sender's name and/or contact information.
- Mentions a company name.

Instructions:

1. Evaluate the email against each criterion.
2. Categorize the message content and assess its legitimacy.
3. Calculate the total score out of 100 based on the point system above.
4. Convert the score to a 1-10 scale.
5. Provide the final relevancy score, category, and a brief explanation in JSON format.

Output the result in the following JSON format:

```
{
  "relevancy_score": <score>,
  "category": "<message content category>",
  "explanation": "<brief explanation of the main factors influencing the score and category>"
}
```

Ensure the output is valid JSON that can be parsed programmatically.

”

Research on what correlates with a successful prompt

Add Contextual Weights:

- Introduce a scoring system that weighs different factors contributing to the overall decision.

- Factors like message relevance and profile credibility can have different weights based on their importance.

Conditional Logic for Special Cases:

- Implement conditional logic to handle special cases where the usual rules might not apply.
- For instance, a relevant message with a high spam score can be flagged for manual review instead of automatic categorization.

Refine JSON Structure for Input and Output:

- Ensure the input JSON for both message and profile analysis GPT models is well-structured and contains all necessary information.
- Define a clear schema for the output JSON to standardize how results are conveyed from both models.